International Student Transition & Career Intelligence Platform

System Construction Roadmap (V2)

1. Our Core Engineering Philosophy

To guarantee maximum system stability and eliminate technical debt, our development follows a strict "Query-First, Scale-Later" iterative loop. We completely validate and master one data layer before advancing to the next.

- **1. BUILD IT:** Create Layer -> Write migration and deploy the tables.
- **2. USE IT:** Write & Verify All Queries -> Complete all CRUD, search, and filter queries.
- **3. CLEAN IT:** Abstract Repetitive Code -> Refactor only when you see real duplication.
- **4. REPEAT:** Advance Layer -> Move to the next migration phase only when the current layer is perfect.

Why we build this way:

- **Zero Blind Building:** We never write database tables without immediately writing and testing the backend queries that power them.
- **Organic Optimization:** We only build codebase abstractions when real-world repetition appears, keeping our software lightweight, elegant, and intentional.
- **Flawless Debugging:** By verifying our data streams incrementally, bugs are isolated instantly to the current phase, preventing systemic errors down the road.

2. Phase Directory & Progress Tracker

Phase 1: Project Setup

- **Status:** COMPLETE
- **Objective:** Establish a stable Command Line Interface (CLI) and PostgreSQL backend foundation.
- **System Components Built:**
    - Directory architecture, global configurations, .env management, and logging engine.
    - Natively managed postgres infrastructure featuring connection.py and executor.py.
- **Execution Flow:** main -> input workflow -> query -> executor -> connection

Phase 2: Migration 001 - Root Tables

- **Status:** COMPLETE

- **Objective:** Construct isolated master entities containing zero external dependencies.
- **Database Tables:** users, universities, companies, document_types, expense_categories
- **Engine Features:** Interactive CLI menus, strict terminal data validation, CRUD handlers, and automated executor integration.
- **Implemented Queries:**
    - *Standard:* create, get_by_id, get_all, update, delete, exists
    - *Utility:* search_users(), search_companies(), filter_universities(), count_users(), system summaries.

Phase 3: Migration 002 - Secondary Master Tables

- **Status:** COMPLETE (Generation 3 Architecture)
- **Objective:** Deploy the first operational layer bound by parent table dependencies.
- **Database Tables:** user_portfolio, programs, intakes, jobs
- **Structural Abstractions Added:**
    - _execute_abstract_input_workflow() engine.
    - Dynamic terminal display helpers, validation suites, and BackSignal menu navigation logic.
- **Implemented Queries:**
    - *Parent-Relative:* get_jobs_by_company(), search_program(), filter_portfolio(), contextual lookup summaries.

Phase 4: Query Expansion Sprint

- **Status:** NEXT FOCUS (NO NEW TABLES)
- **Objective:** Maximize data extraction value and master complex lookup architectures before scaling the physical schema.
- **Core Learning Focus:** Dynamic SQL generation, multi-parameter filtering, and algorithmic query composition.
- **Target Queries to Build:**
    - search_users() & search_jobs()
    - filter_programs() & latest_records()
    - Comprehensive cross-entity search filters and dataset counters.

Phase 5: Migration 003 - Workflow Tables

- **Status:** PLANNED
- **Objective:** Map relational processes and track real-time applications using multi-parent intersections.
- **Database Tables:** university_applications, job_applications
- **Milestone:** Introduction of our first deep SQL JOIN dependencies.
- **Target Queries to Build:**
    - get_user_applications(), search_application(), filter_status(), move_status(), pipeline_summary()

Phase 5 Repeat: Migration 004 - Operational Tables

- **Status:** PLANNED
- **Objective:** Introduce automated system lifecycles, time-series calendars, and strict documentation pipelines.
- **Database Tables:** documents, calendar_events
- **Target Queries to Build:**
    - expired_documents(), upcoming_events(), events_by_date(), document_summary()
    - Advanced timezone-aware date operations and linear timeline queries.

Phase 5 Repeat: Migration 005 - Aggregation Ledger

- **Status:** PLANNED
- **Objective:** Deploy the unified financial ledger to intersect cost data across every asset class in the network.
- **Database Tables:** journey_expenses
- **Target Queries to Build:**
    - expense_summary(), monthly_cost(), cost_per_application(), complex financial analytics dashboards.
    - Heavy multi-join aggregation algorithms.

Phase 6: Workflow Layer

- **Status:** PLANNED (NO NEW TABLES)
- **Objective:** Turn passive database records into reactive backend system behaviors.

- **Target Queries to Build:**
    - get_user_dashboard(), get_next_action(), get_pending_items(), journey_progress(), deadline_overview()
    - Unified aggregate data streams spanning applications, documents, events, and costs.

Phase 7: System Intelligence Layer

- **Status:** PLANNED (FINAL DATABASE PHASE)
- **Objective:** Convert raw historical transaction metrics into actionable predictive insights.
- **Target Queries to Build:**
    - monthly_summary(), application_success(), completion_score(), overall_progress()
    - **Final Output Engines:** Highly advanced metrics covering Education, Career, Finance, and Global Operations.

3. Macro-Execution Timeline Overview

Plaintext

1. [Setup Engine]
2. [Root Master Tables]
3. [Secondary Dependent Tables]
4. [Query Expansion Sprint]
5. [Workflow Process Trackers]
6. [Operational Calendars & Docs]
7. [Unified Capital Ledgers]
8. [Unified User Dashboards]
9. [Intelligence Analytical Engine] -> (Database + Backend Construction Complete)